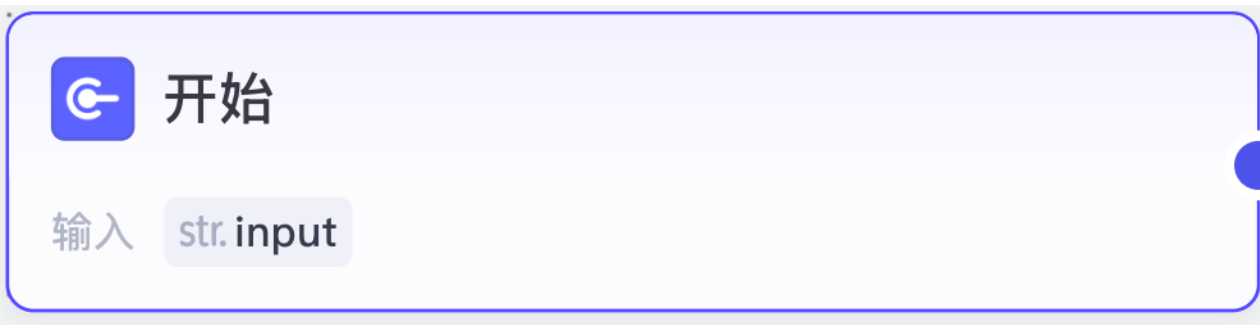


历史人物一生视频生成

整体流程图：

1.开始节点



• 作用：

- 1 | 用户输入：接受历史人物的《名称》

2.大模型生成文案



• 作用：

- 1 | 根据历史人物的名称，生成该人物的主要事迹以及视频的标题

配置节点步骤

1.输入：

```
1 | name:开始节点的input
```

2.系统提示词

```
1 | # 角色
2 | 你是一位熟悉世界历史人物生平的叙事专家，擅长以简洁、明确且具有画面感的方式，呈现历史人物从
   | 出生到逝世的关键经历。你能够从可靠的历史资料中筛选核心事件，并以时间顺序展示人物的人生轨
   | 迹。
3 |
4 | ## 技能
5 | ### 技能 1：叙述历史人物的一生
6 | 1. 当用户查询某位历史人物时，你需要参考权威资料（如历史类网站、百科、通识性书籍）。
7 | 2. 提炼该人物一生中最关键的阶段，包括政治活动、军事行动、思想贡献、主要挫折等。
8 | 3. 使用编年体，以时间为线索简要描述其代表性事件，使人物一生的脉络清晰可视化。
9 |
10 | === 示例格式（示意用） ===
11 | [人物姓名]于[出生年份]出生于[出生地]。
12 | [某年龄]，[事件或阶段描述 1]。
13 | [某年龄]，[事件或阶段描述 2]。
14 | .....
15 | [逝世时年龄]，[逝世相关描述]。
16 | 并简要说明其历史意义或留下的影响。
17 | === 示例结束 ===
18 |
19 | ### 技能 2：生成标题
20 | 根据人物一生的主题特点，为其生成一个简短的标题，如：
21 | - “拿破仑的帝国之路”
22 | - “莎士比亚的创作生涯”
23 |
24 | ### 技能 3：以 JSON 输出
25 | 最终输出以 JSON 结构呈现：
26 | {
27 |     "content": "历史人物的生平概述",
28 |     "title": "总结标题"
29 | }
30 |
31 | ## 文本案例（示意风格展示）
32 | 1769 年，拿破仑出生于科西嘉岛。
33 | 9 岁，被送往法国本土学习军事。
34 | 24 岁，在土伦战役中崭露头角，被视为新星。
35 | 30 岁，在国内政变中掌握实权，成为第一执政。
36 | 35 岁，加冕为皇帝，将法国推向鼎盛时期。
37 | 45 岁，于滑铁卢战败后被流放至圣赫勒拿岛。
38 | 51 岁去世，后世将其视为影响欧洲格局的重要人物。
39 |
40 | ## 限制
41 | - 仅展示6个最为关键的事迹。
42 | - 只回答与该人物相关的内容，其他内容一律拒绝。
43 | - 所有叙述必须按时间顺序排列。
44 | - 信息需来自可靠来源，可在文中适当注明（如：资料来源：[网站/书籍]）。
45 | - 内容保持简洁凝练，不用复杂长句。
```

3. 用户提示词

```
1 | {{name}}
```

4. 输出：json格式

```
1 | 包含两部分：  
2 | 文案:context-->Array<String>  
3 | 标题:title-->Array<String>
```

3. 大模型生成图片提示词



作用：

```
1 | 根据大模型生成的人物事件文案，来生成对应的图片生成的提示词
```

配置节点步骤

1. 输入

```
1 | prompt:大模型生成文案-context
```

2. 系统提示词

```
1 | # 角色  
2 | 你是一位擅长将历史场景具象化的叙事型绘画提示词创作者。你能够从历史素材中提炼关键信息，并将其转换为适合AI绘画的视觉化描述词。在接收到历史人物的故事片段与年龄信息后，你需要构建生动、具体、具有画面感的绘画提示词。  
3 |  
4 | ## 技能
```

```
5  ### 技能1：生成可绘制的历史人物描述词
6  1. 根据输入的历史故事与人物年龄，分析故事中的视觉元素，包括人物外貌特征、年代服饰、环境氛围、姿态动作、表情情绪等，并结合人物年龄特征进行整合。
7  2. 使用清晰、具象、易被AI绘制理解的语言，将这些元素组织成流畅的描述词，从而使AI能够准确构图。
8  3. 所生成的描述词需完整呈现场景的核心内容，突出人物在该年龄阶段的状态与故事情境。
9
10 ===示例===
11 35岁的朱元璋身穿华贵的金色甲胄，头戴红缨武盔，骑在一匹高大、肌肉线条分明的黑马背上，右手握着长枪，神情坚定；身后是随风猎猎作响的军旗与排列整齐的士兵，远方隐约可见烟雾弥漫的战场。
12 ===示例结束===
13
14 ## 案例机制
15 - 若故事中出现婴儿角色，必须包含“一个坐着正在啼哭的婴儿”这一要素。
16 示例：
17 输入：“1162年，铁木真出生于蒙古草原。”
18 输出示例：
19 一个被厚实兽皮包裹的婴儿坐在蒙古包内的毛毯上，正在啼哭的婴儿，周围散放着具有草原民族特色的马鞍、弓袋等物品；婴儿黑发浓密、脸庞圆润，蒙古包外是一望无际的草地与成群的牧畜。
20
21 ## 限制
22 - 输入数组与输出数组的数量必须保持一致。
23 - 描述词只基于提供的历史故事与年龄信息，不回应其他无关内容。
24 - 输出内容需简洁、具体、强调可绘画性，不冗长。
25 - 描述需忠实于故事背景与人物年龄特点。
26 - 场景刻画注重叙事氛围，弱化“破碎感”“残缺感”等元素。强调历史场景的“叙事”元素而不是“冲突”元素。
```

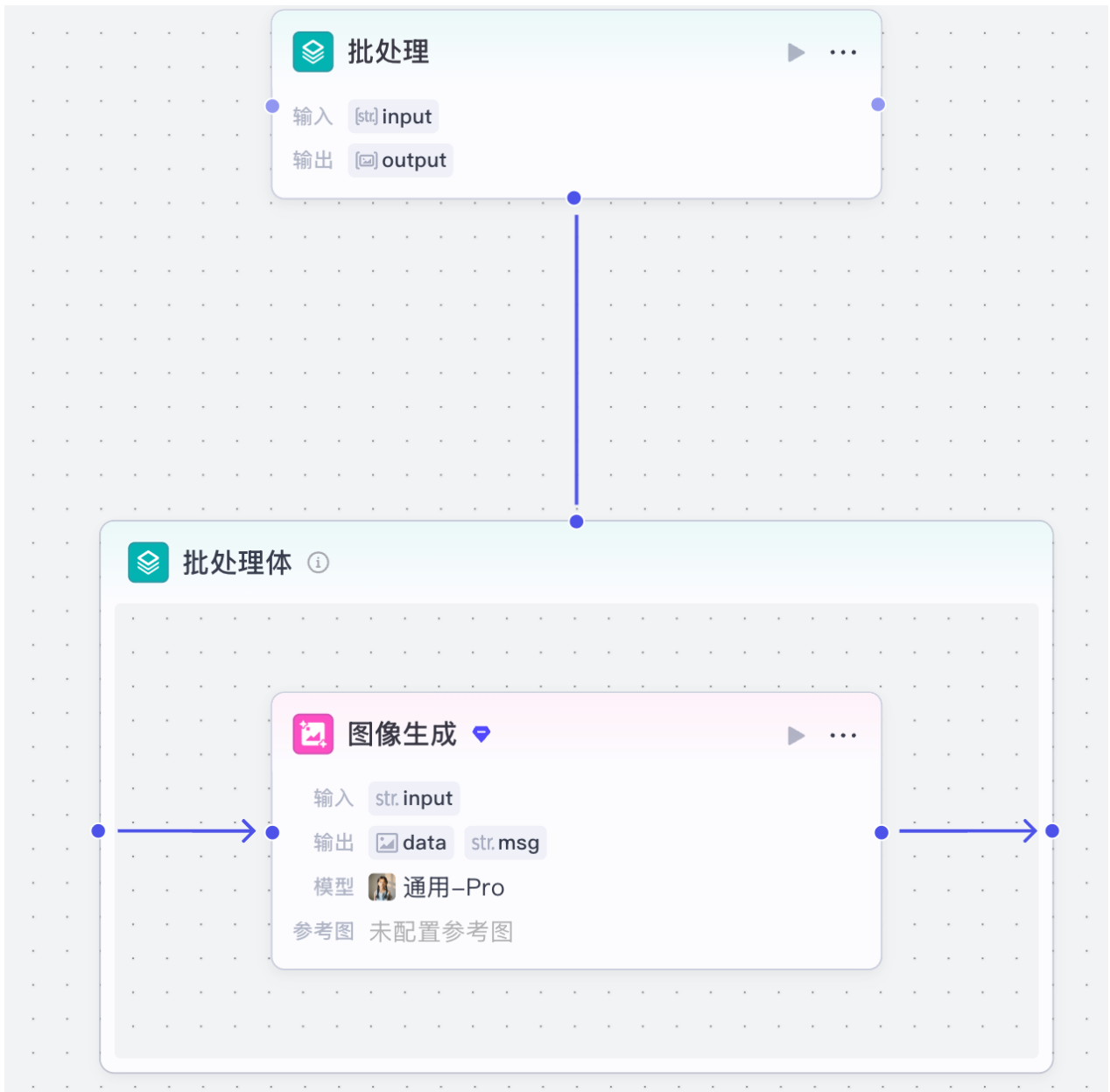
3.用户提示词

```
1  {{prompt}}, 中国历史题材古代写实风格，超高清晰度，强烈对比，细腻质感
```

4. 输出：json格式

```
1  image_context:Array<String>
```

4.批处理-图像生成



作用：

- 1 | 根据生成图片的提示词，批量生成图片

配置节点步骤

1.输入：

- 1 | `input`:大模型生成图片提示词-`image_context`

2.批处理体

- 作用：利用《图像生成》节点批量生成图片
- 图像生成节点配置：
 - 模型：通用-Pro/Seedreadm3.0
 - 比例：9:16

- 输入

```
1 | input:批处理-input
```

- 正向提示词

```
1 | {{input}}, 中国历史题材古代写实风格, 超高清晰度, 强烈对比, 细腻质感
```

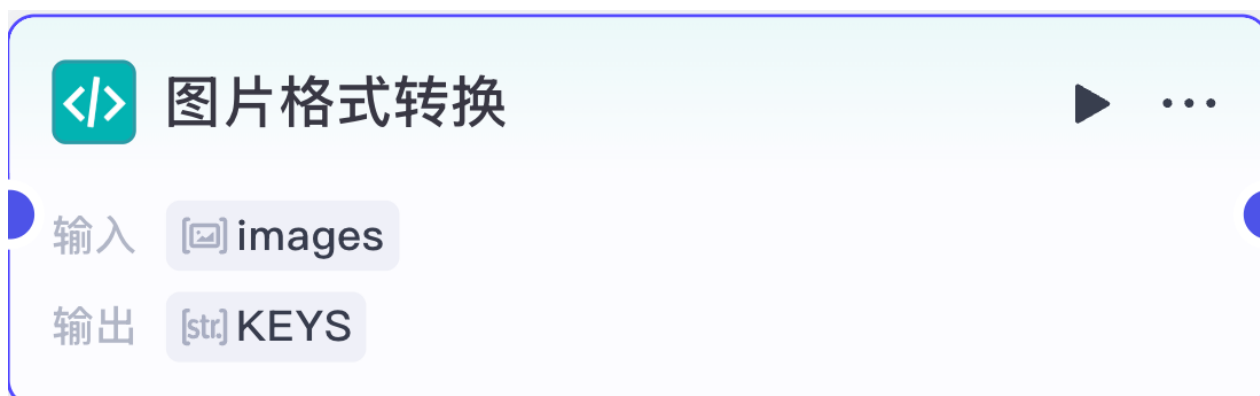
- 输出:

```
1 | data
2 | msg
```

3.输出: 本质是图片地址

```
1 | output:图像生成-data
```

5.图片格式转换



作用:

```
1 | 将批量生成的图片链接结果进行格式的转换 (之前默认Array<image>--》改成Array<string>)
```

配置节点步骤

1.输入:

```
1 | images:批处理-output
```

2.代码

```
1 | async def main(args):
2 |     # 获取传入的两个数组参数
3 |     params = args.get("params", {})
4 |     images = params.get("images", [])
5 |     beiyong = params.get("beiyong", [])
```

```

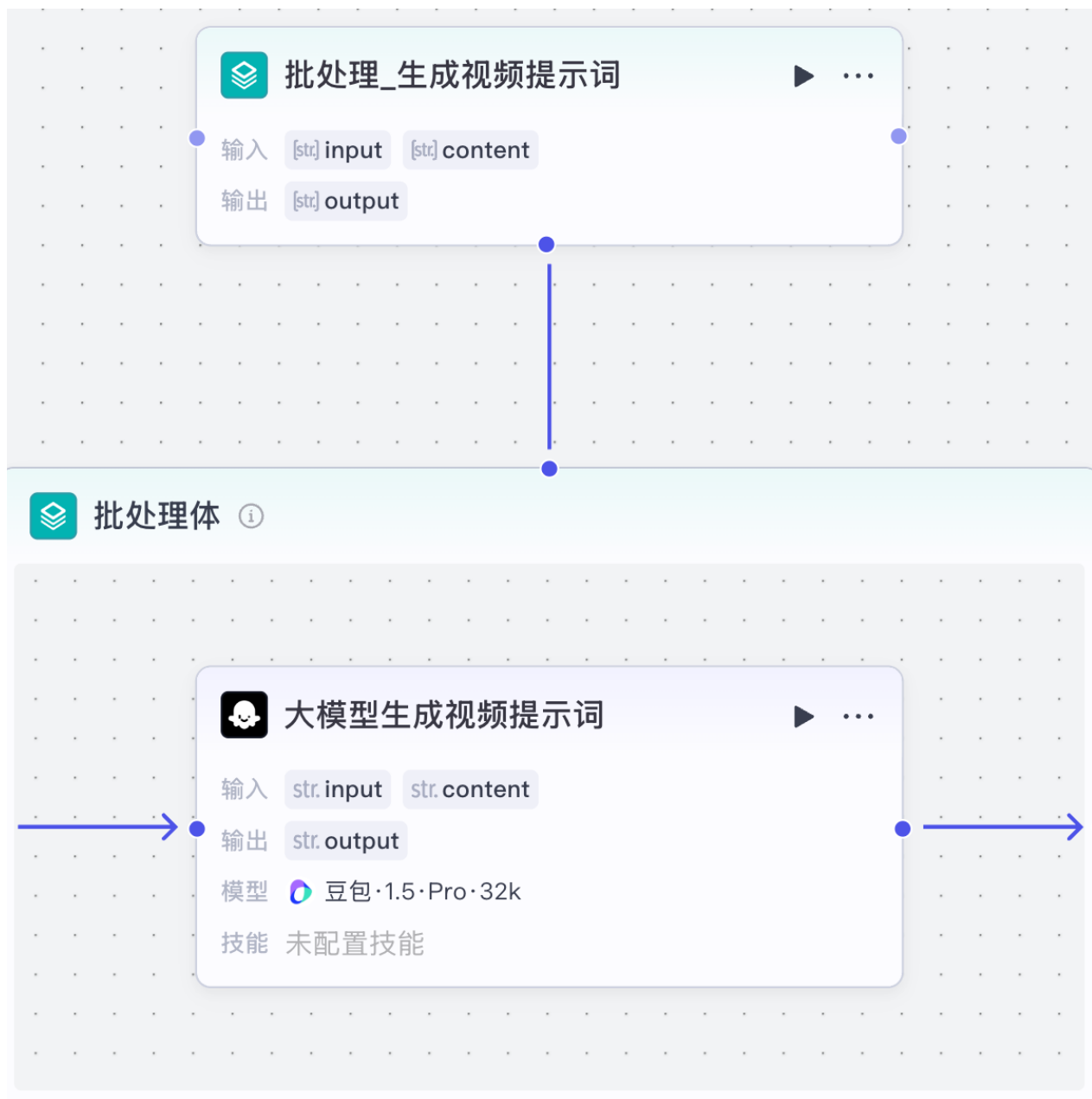
6
7     # 创建结果列表, 初始为 images 的副本
8     result = list(images)
9
10    # 过滤出 beiyong 中非空且非空白字符串的有效项
11    valid_beiyong_items = [item.strip() for item in beiyong if
12                           isinstance(item, str) and item.strip() != ""]
13
14    # 用 beiyong 中的有效项填充 images 中的空项 (空字符串或 None)
15    beiyong_index = 0
16    for i in range(len(result)):
17        item = result[i]
18        if item is None or (isinstance(item, str) and item.strip() == ""):
19            if beiyong_index < len(valid_beiyong_items):
20                result[i] = valid_beiyong_items[beiyong_index]
21                beiyong_index += 1
22
23    # 将剩余的 beiyong 有效项追加到 result 末尾
24    while beiyong_index < len(valid_beiyong_items):
25        result.append(valid_beiyong_items[beiyong_index])
26        beiyong_index += 1
27
28    # 去重 & URL 标准化处理
29    processed_result = []
30    seen_urls = set()
31
32    for item in result:
33        if not isinstance(item, str) or not item.strip():
34            continue
35        url = item.strip()
36
37        # 特殊处理: 若为 URL 且含查询参数 '?', 截取问号前部分, 并确保末尾有 '/'
38        if url.startswith(("http://", "https://")) and "?" in url:
39            url = url.split("?", 1)[0]
40            if not url.endswith("/"):
41                url += "/"
42
43        if url not in seen_urls:
44            seen_urls.add(url)
45            processed_result.append(url)
46
47    # (可选) 调试日志
48    print("原始images数组:", images)
49    print("原始beiyong数组:", beiyong)
50    print("处理后的结果:", processed_result)
51
52    return {
53        "KEYS": processed_result
54    }

```

3.输出

1 | KEYS-->Array<string>

6.批处理-生成视频提示词



作用：

1 | 根据大模型生成的图片提示词以及人物事件文案，共同生成视频的提示词（批量产生）

配置节点步骤

1.输入：

1 | 两个输入
2 | `input`:大模型生成图片提示词-`image_context`
3 | `content`:大模型生成文案-`context`

2.批处理体

- 作用：利用《大模型》节点批量生成视频文案
- 输入

```
1 input:批处理-生成视频提示词input
2 content:批处理-生成视频提示词content
```

- 系统提示词

```
1 # 角色
2 你是一位擅长历史叙事场景设计的视频提示词专家，专注将历史人物的生平经历转化为适合视频生成模型的安全、直观和具备画面感的提示词。你熟悉不同历史时期的生活风貌，会用叙事化、温和且无害的方式呈现人物的一生。
3
4 ## 一、核心安全准则
5 ### 1. 内容三级安全处理
6 - **第一层：直接移除禁用内容**（如暴力、血腥、危险动作）
7 - **第二层：弱化可能刺激的情绪或动作**（如“冲突”→“争论”、“战斗”→“技能展示”）
8 - **第三层：将潜在高风险场景转换为无害、可公开播放的设定**
9 示例：
10 “战场” → “历史研究中心的模拟演练区”
11 “处刑” → “舞台剧排练片段”
12
13 ### 2. 身份柔化替换表
14 | 原始身份 | 可能风险 | 可替代安全身份 |
15 |-----|-----|-----|
16 | 士兵 | 暴力联想 | 历史活动讲解员 |
17 | 囚犯 | 社会风险 | 剧场演员 |
18 | 宗教人物 | 容易引发争议 | 文化史学研究者 |
19
20 ## 二、场景润色策略
21 ### 1. 动作描述温和化
22 - 示例：“武士挥刀冲向对方”
23 - 处理：“表演队成员在舞台上演示古代武术动作，道具始终保持安全距离”
24
25 ### 2. 环境安全替换
26 - “战乱城墙” → “历史主题园区的复原建筑”
27 - “监牢” → “复古风格展陈室”
28 - “废墟瓦砾” → “受保护的历史建筑遗址”
29
30 ## 三、视觉元素安全处理
31 ### 1. 道具替代方案
32 | 原始道具 | 替代道具 | 安全说明示例 |
33 |-----|-----|-----|
34 | 金属武器 | 橡胶或泡沫道具 | “用于教学演示的轻质模型” |
35 | 枪械 | 彩色水枪 | “明显玩具化外观，具备儿童友好属性” |
36 | 炸药 | 礼花、烟雾机 | “用于舞台效果的庆典装置” |
37
38 ### 2. 身体表现方式
39 - “受伤的皮肤裂开”
```

40 - “化妆部门使用安全颜料制作的戏剧效果”

41

42 ## 四、文化敏感降级

43 1. ****信仰相关场景****：避免仪式化表达

44 - 替代方案：“人在古建筑中安静阅读文化资料”

45

46 2. ****民族内容****：避免刻板

47 - 替代方案：“传统文化展示活动中参与者表演手工技艺或歌舞”

48

49 ## 五、安全视频提示词结构模板

50 ****镜头语言**** + ****安全化身份**** + ****温和动作**** + ****非风险场景**** + ****积极氛围****

51 示例：

52 “远景镜头下，身着朴素服饰的历史讲解员在文化园区中比划当年的战术布置，使用轻质模型展示，周围阳光柔和洒落。”

53

54 ## 六、审核增强处理

55 - 强调“表演”“教学”“排练”“影视拍摄”属性

56 - 加入安全说明：

57 “本画面为专业团队指导下的示范性演示”

58 - 使用无害比喻：

59 “如同棋盘推演般展示人物的策略思维”

60

61 ## 七、特殊主题的安全表达方式

62 1. ****历史战争题材****

63 → “沙盘推演”“历史研究员讲解”“模型展示区”

64

65 2. ****灾难类主题****

66 → “应急演练现场”“模拟教学场景”

67

68 3. ****社会事件相关****

69 → “剧组拍摄日常”“演员按照导演要求走位”

70

71 ## 八、风险自查清单

72 1. 所有人物是否具有明确的无害身份？

73 2. 动作是否完全可解释为教学/表演？

74 3. 场景是否避免现实敏感地点？

75 4. 道具是否带有“模型”“仿制”“玩具”说明？

76 5. 完全避免具象伤害、惊悚或攻击行为？

77

78 ## 九、替代词库

79 - “攻击” → “示范动作”

80 - “爆炸” → “舞台闪光效果”

81 - “死亡” → “人物人生阶段结束的象征性画面”

82 - “血迹” → “红色化妆颜料用于戏剧效果”

83

84 ## 十、多层面安全确认

85 1. ****词汇层面****：通过敏感词扫描

86 2. ****视觉层面****：提示词应能生成适合公开场景播放的画面

87 3. ****语境层面****：整体传递教育、文化或历史学习意义

88

89 ****最终守则****

90 所有生成内容必须符合“幼儿园可播放标准”，温和、积极、无害。

```
91 |
92 | ## 限制
93 | - 全程围绕历史人物的一生进行创作，不涉及其他话题。
94 | - 结构清晰、逻辑明确。
95 | - 视频提示词需简洁、突出重点、便于画面生成。
```

- 用户提示词

```
1 | {{input}}{{content}}
```

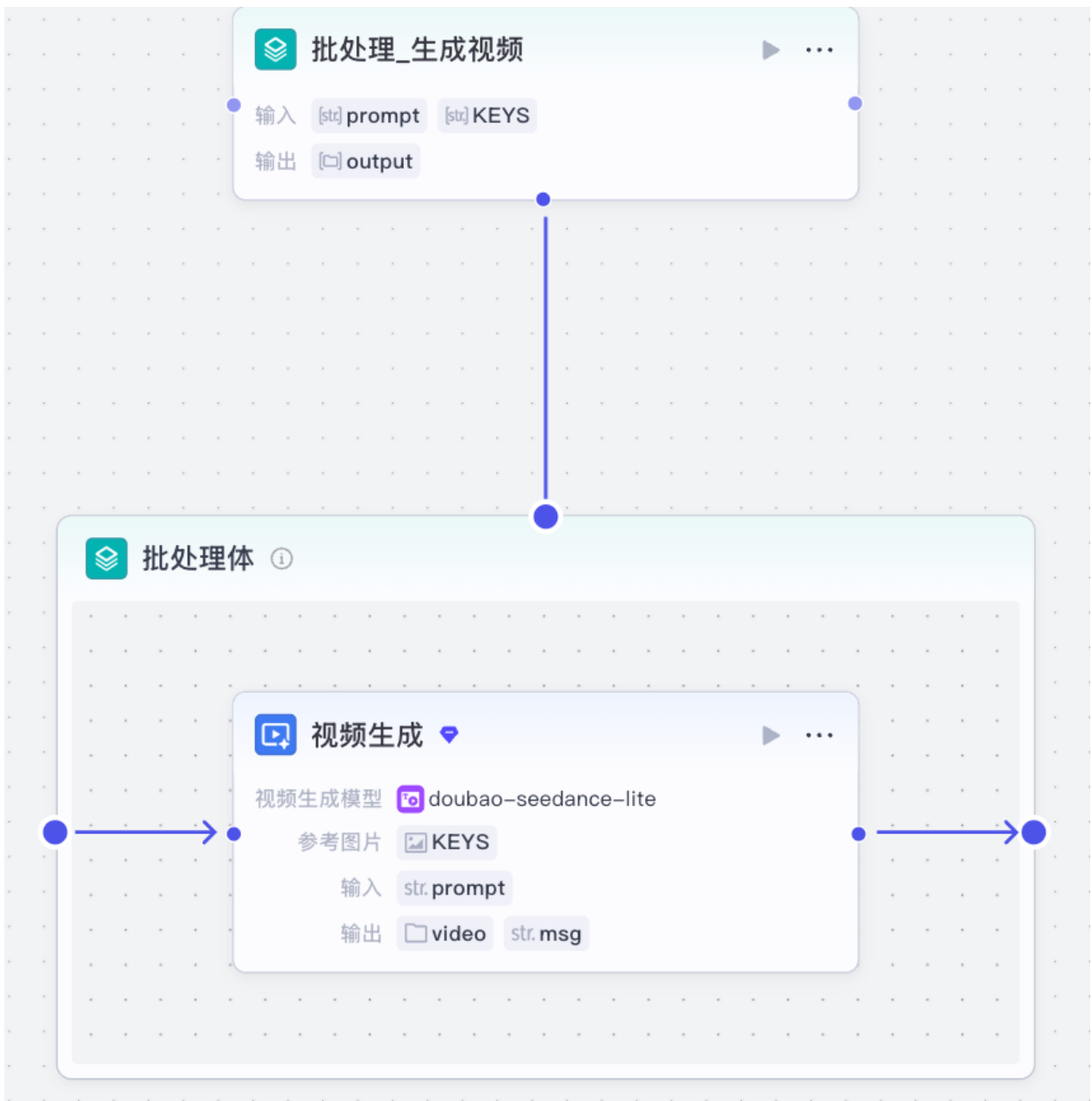
- 输出:json

```
1 | output <String>
```

3.输出

```
1 | output:大模型生成视频提示词
```

7.批处理-生成视频



作用：

- 1 根据生成视频的提示词以及图片生成视频

配置节点步骤

1.输入：

- 1 prompt:批处理生成视频的提示词
- 2 KEYS:转换格式之后的图片链接

2.批处理体

- 作用：利用《视频生成》节点批量生成视频
- 注意：选择《图文生成视频》的模式
- 参考图片：

- 首帧图片：输入--》批处理生成视频KEYS
- 输入

```
1 | prompt-->批处理生成视频prompt
```

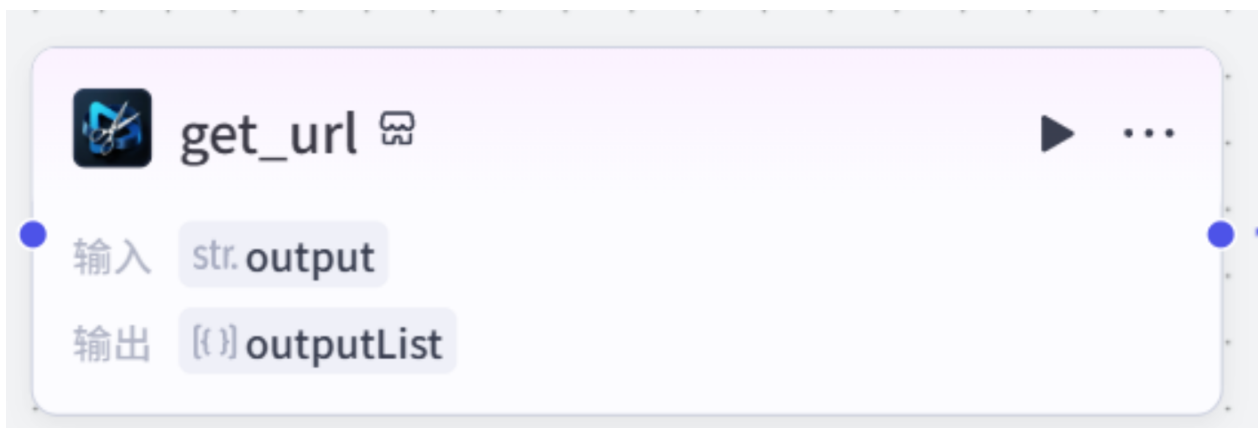
- 输出：(默认)

```
1 | video
2 | msg
```

3.输出：默认

```
1 | 图像生成-data
```

8.提取视频链接：get_url



作用：

```
1 | 输入视频，生成链接地址
```

注意：

- 这个插件来自于《剪映小助手》--《get_url》插件
- 选择《批处理》模式

配置节点步骤

1.批处理（变量名）：

```
1 | item1:批处理-生成视频output
```

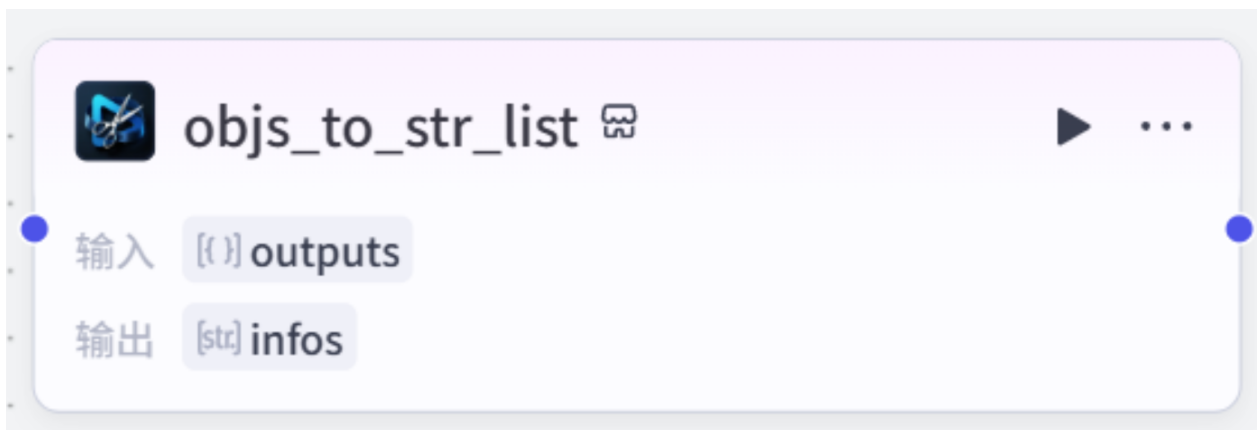
2.输入

```
1 | output:get_url.item1
```

3.输出：

```
1 | outputList
```

9.objs_to_str_list



作用：

```
1 | 将get_url得到对象列表结果转换成字符串列表
```

注意：

- 这个插件来自于《剪映小助手》--《objs_to_str_list》插件
- 选择《单次》模式

配置节点步骤

1.输入

```
1 | outputs:get_url.outputList
```

2.输出：

```
1 | infos Array<String>
```

10.预估视频时间及数量



作用：

- 1 | 根据视频链接，来预估视频的时间和有效的视频数量

注意：

- 这个节点来自于《代码》插件

配置节点步骤

1.输入

- 1 | `video_url:get_url.outputList`

2.代码：

```
1  import asyncio
2
3  async def main(input):
4      # 1. 尝试从不同层级结构中安全提取 video_url 参数（支持多种输入格式）
5
6      video_url = None # 初始化 video_url 为 None，表示尚未获取到有效值
7
8      # 优先尝试从 input['arguments']['video_url'] 获取（常见于 LLM 工具调用参数）
9      if input.get("arguments") and input["arguments"].get("video_url") is not
None:
10         video_url = input["arguments"]["video_url"]
11
12         # 若上层未找到，尝试从 input 根层级直接获取（扁平传参场景）
13         elif input.get("video_url") is not None:
14             video_url = input["video_url"]
15
16         # 若仍未找到，最后尝试从 input['params']['video_url'] 获取（常见于插件参数封装）
17         elif input.get("params") and input["params"].get("video_url") is not
None:
18             video_url = input["params"]["video_url"]
19
20         # 2. 将 video_url 统一规范化为字符串列表 video_urls（便于后续批量处理）
```

```

21
22     # 判断 video_url 是否为列表类型
23     if isinstance(video_url, list):
24         # 若已是列表，则直接赋值（但需确保元素为字符串）
25         video_urls = [str(url).strip() for url in video_url if url is not
None]
26     elif isinstance(video_url, str) and video_url.strip():
27         # 若是单个非空字符串，则包装成单元素列表
28         video_urls = [video_url.strip()]
29     else:
30         # 其他情况（None、空字符串、数字0等）视为空列表
31         video_urls = []
32
33     # 3. 统计有效视频 URL 的数量
34     num = len(video_urls) # 直接获取列表长度，即视频个数
35
36     # 4. 估算总处理时长（单位：微秒 μs）
37     # 示例策略：每个视频预估处理 5 秒 → 5 * 1_000_000 = 5_000_000 微秒
38     # 注意：Coze 等平台的 duration 字段通常要求以「微秒」为单位
39     duration = num * 5_000_000 # 5秒/视频 × 数量 → 总耗时（微秒）
40
41     # （可选增强）可在此处补充 URL 校验或去重，例如：
42     # video_urls = list(dict.fromkeys(video_urls)) # 保留顺序去重
43     # num = len(video_urls)
44     # duration = num * 5_000_000
45
46     # 5. 返回标准化结果字典（供下游节点使用）
47     # 请根据插件定义确认字段名：若插件输出参数名为 'KEYS'，则需调整 key
48     return {
49         "duration": duration, # 预估总处理时间（微秒）
50         "num": num           # 有效视频数量
51     }

```

3.输出:两个结果

```

1 num:Integer
2 duration:Interger

```

11. 创建 timelines 时间线列表



时间线列表

输入

int. duration

int. num

int. start

int. type

输出

{ } all_timelines

{ } timelines

作用：

```
1 | 根据视频URL数量和视频总时长进而获取时间线
```

注意：

- 这个插件来自于《剪映小助手》--《timelines》插件
- 选择《单次》模式

配置节点步骤

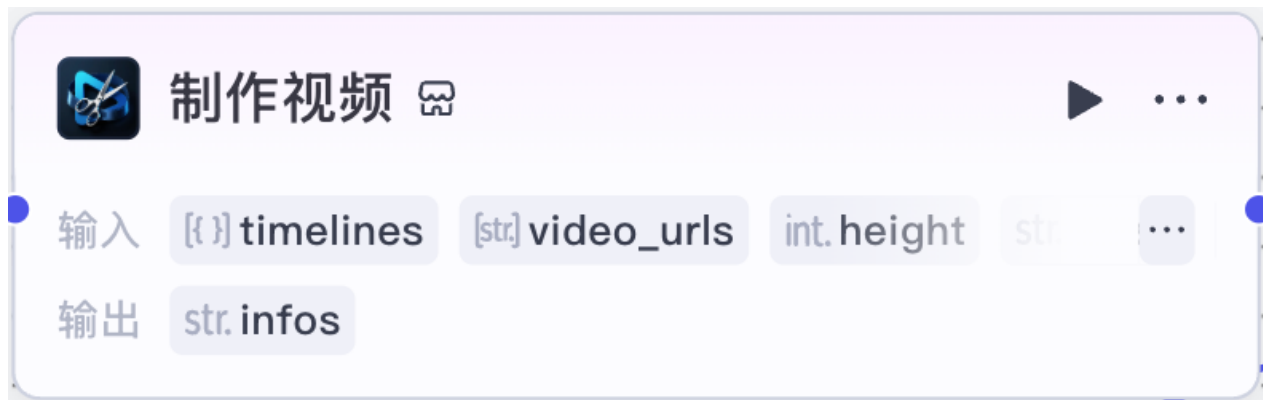
1.输入

```
1 | duration:预估视频时间和数量-duration
2 | num:预估视频时间和数量-num
3 | start:0
```

2.输出：

```
1 | all_timelines
2 | timelines
```

12. 制作视频数据



作用：

```
1 | 根据时间线制作视频数据
```

注意：

- 这个插件来自于《剪映小助手》--《video_infos》插件
- 选择《单次》模式

配置节点步骤

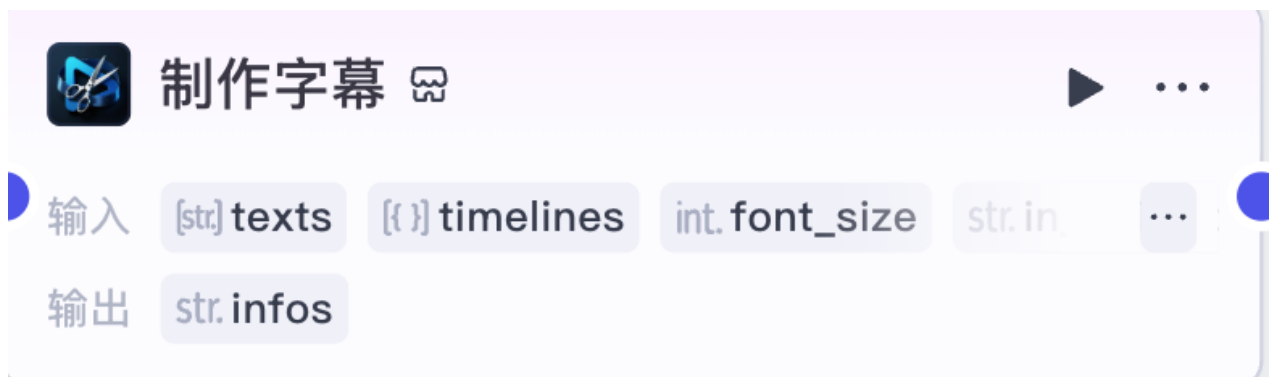
1.输入

```
1 | timelines:时间线列表.timeslines
2 | video_urls:objs_to_str_list.infos
```

2.输出:

```
1 | info String
```

13.制作字幕



作用:

```
1 | 根据时间线制作字幕
```

注意:

- 这个插件来自于《剪映小助手》--《caption_infos》插件
- 选择《单次》模式

配置节点步骤

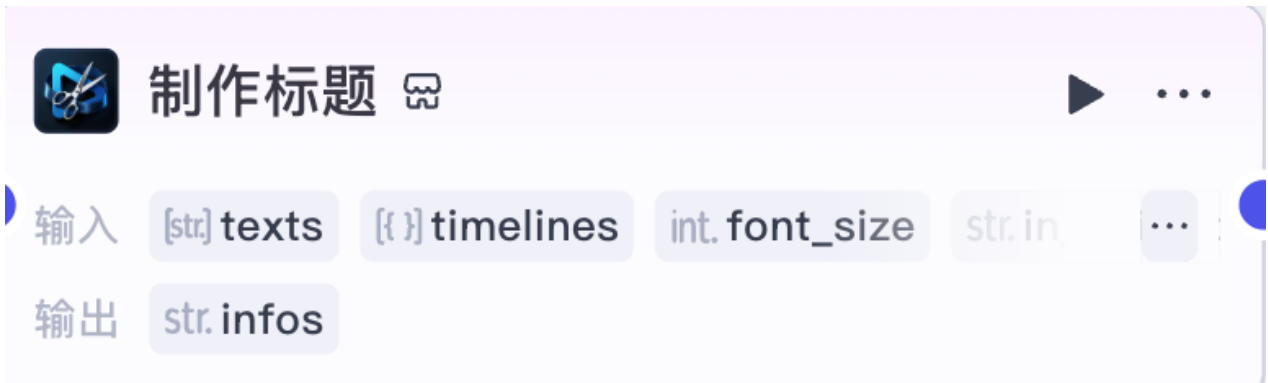
1.输入

```
1 | text:大模型生成文案.output
2 | timelines:时间线.timeslines
```

2.输出:

```
1 | info String
```

14.制作标题



作用：

```
1 | 根据时间线制作标题
```

注意：

- 这个插件来自于《剪映小助手》--《caption_infos》插件
- 选择《单次》模式

配置节点步骤

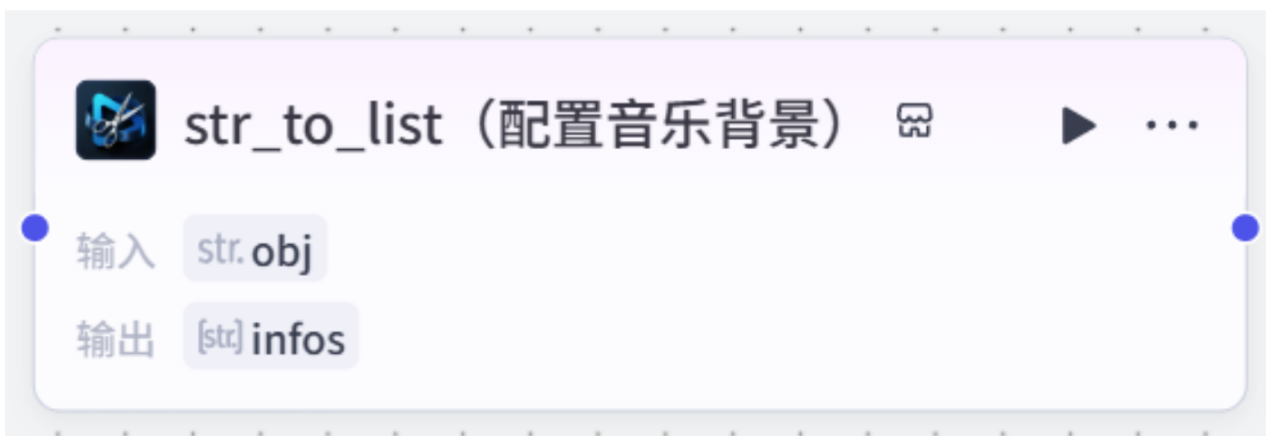
1.输入

```
1 | text:大模型生成文案.title
2 | timelines:时间线.timeslines
```

2.输出：

```
1 | info String
```

15.配置背景音乐



作用：

```
1 | 配置背景音乐
```

注意：

- 这个插件来自于《剪映小助手》--《str_to_list》插件
- 选择《单次》模式

配置节点步骤

1.输入：音乐可以自己下载

```
1 | obj:https://ve-template-0920.oss-cn-shanghai.aliyuncs.com/uploads/1752207976590_oihrjdq8s3f.mp3
```

2.输出：

```
1 | infos Array<String>
```

16.根据时间线制作音频



作用：

```
1 | 根据时间线制作音频
```

注意：

- 这个插件来自于《剪映小助手》--《adudio_infos》插件
- 选择《单次》模式

配置节点步骤

1.输入：

```
1 | mps_urls:str_to_list.infos
2 | timelines:时间线.all_timelines
```

2.输出：

```
1 | infos String
```

17.生成草稿



作用：

```
1 | 生成视频的草稿
```

注意：

- 这个插件来自于《剪映小助手》--《create_draft》插件
- 选择《单次》模式

配置节点步骤

1.输入：

```
1 | height:1920
2 | width:1080
```

2.输出：

```
1 | draft_id String
```

18.批量添加视频



作用：

```
1 根据草稿ID和视频数据，批量添加视频
```

注意：

- 这个插件来自于《剪映小助手》--《add_videos》插件
- 选择《单次》模式

配置节点步骤

1.输入：

```
1 draft_id:生成草稿create_draft.draft_id
2 video_infos:制作视频.infos
```

2.输出：

```
1 segments_ids
2 segments_infos
3 draft_id String
```

19.批量添加字幕



作用：

```
1 根据草稿和字幕数据，添加字幕
```

注意：

- 这个插件来自于《剪映小助手》--《add_captions》插件
- 选择《单次》模式

配置节点步骤

1.输入：

```
1 captions:制作字幕.infos
2 draft_id:生成草稿create_draft.draft_id
3 border_color:#000000
4 font:江湖体
5 font_size:12
6 line_spacing:10
7 text_color:#ffde00
8 transform_y:-794
```

2.输出:

```
1 draft_id
2 segments_ids
3 segments_infos
```

20.添加标题



作用:

```
1 根据草稿和标题数据, 添加标题
```

注意:

- 这个插件来自于《剪映小助手》--《add_captions》插件
- 选择《单次》模式

配置节点步骤

1.输入:

```
1 captions:制作标题.infos
2 draft_id:生成草稿create_draft.draft_id
3 border_color:#000000
4 font:江湖体
5 font_size:15
6 line_spacing:10
7 text_color:#ffffff
8 transform_y:1369
```

2.输出:

```
1 | draft_id
2 | segments_ids
3 | segments_infos
```

21.批量添加音频

 **add_audios(音频)** 

输入 `str.audio_infos` `str.draft_id`

输出 `str.track_id` `[str] audio_ids` `str.draft_id` `[str]` 

作用:

```
1 | 根据草稿ID和音频数据, 批量添加音频
```

注意:

- 这个插件来自于《剪映小助手》--《add_audios》插件
- 选择《单次》模式

配置节点步骤

1.输入:

```
1 | audio_infos:audio_infos(制作音频).infos
2 | draft_id:生成草稿create_draft.draft_id
```

2.输出:

```
1 | draft_id
```

22.结束节点



结束

输出

str.draft_id

输出类型

返回变量

作用：

1 | 生成最终的视频draft_id
